

PW EARNERS
DSA in C++

DPP — 1 (SOLUTIONS)

Topic: Basics of C++

Q1. State **True** or **False** for each statement below.

- (i) In C++, a `double` variable can store 3.14, while an `int` would truncate it to 3. **True**
- (ii) The modulus operator `%` works with `double` operands just like with `int` operands. **False**
- (iii) Blank spaces are allowed inside a variable name in C++ (e.g., `my variable` is valid). **False**
- (iv) The post-increment operator (`x++`) first uses the current value in the expression, then increments it. **True**

.....

Q2. Identify whether each of the following is a **valid** or **invalid** C++ variable name. For every invalid name, write a brief reason.

Variable Name	Valid / Invalid	Reason (if invalid)
(i) <code>totalMarks</code>	Valid	
(ii) <code>_counter</code>	Valid	
(iii) <code>2ndPlayer</code>	Invalid	Cannot begin with a digit.
(iv) <code>player#score</code>	Invalid	Special characters like <code>#</code> are not allowed.
(v) <code>int</code>	Invalid	<code>int</code> is a reserved keyword.
(vi) <code>Hello World</code>	Invalid	Spaces are not allowed in identifiers.
(vii) <code>MAX_SIZE</code>	Valid	
(viii) <code>my, var</code>	Invalid	Commas are not allowed.
(ix) <code>__x1</code>	Valid	(Though reserved for system use)
(x) <code>Float</code>	Valid	C++ is case-sensitive; <code>Float</code> $\neq$ <code>float</code> .

.....

Q3. Choose the correct answer for each part.

(i) What is the result of $17 \% 5$ in C++?

A. 3

B. 2

C. 1

D. 0

Solution: B. $17 = 5 \times 3 + 2$. The remainder is 2.

(ii) Which declaration correctly stores 3.14159?

A. `int pi = 3.14159;`B. `char pi = 3.14159;`C. `double pi = 3.14159;`D. `bool pi = 3.14159;`

Solution: C. double handles fractional/floating-point values precisely.

(iii) If `int a = 5, b = 2;`, what does `a / b` give?

A. 2.5

B. 2

C. 3

D. 2.0

Solution: B. Division between two integers truncates the decimal part.

.....

For Q4 to Q10: Write the exact output of each program. If there is a compilation error, write "Compilation Error" and state why.

Q4.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 10;
5     cout << x++ << endl;
6     cout << x << endl;
7     return 0;
8 }
```

Output:

10

11

.....

Q5.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 10;
5     cout << ++x << endl;
6     cout << x << endl;
7     return 0;
8 }
```

Output:

11

11

Q6.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 7, b = 3;
5     cout << a % b << endl;
6     cout << (-7) % 3 << endl;
7     return 0;
8 }
```

Output:

1
-1

Explanation:

- $7 \% 3$: The remainder of 7 divided by 3 is 1.
- $(-7) \% 3$: In C++, the sign of the modulo operator result always matches the sign of the left operand (the dividend). Since -7 is negative, the remainder evaluates to -1.

Q7.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 5;
5     double b = 2.0;
6     cout << a / 2 << endl;
7     cout << a / b << endl;
8     return 0;
9 }
```

Output:

2
2.5

Explanation:

- $a / 2$: Here, a (5) and 2 are both integers. C++ performs *integer division*, discarding the decimal part. So, $5/2$ outputs 2 instead of 2.5.
- a / b : Here, a is an int (5) and b is a double (2.0). C++ automatically converts a to a double for the calculation, keeping the fractional part and giving the exact decimal result, 2.5.

Q8.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     char ch = 'A';
5     cout << ch << endl;
6     cout << (int)ch << endl;
```

```

7     cout << ch + 1 << endl;
8     return 0;
9 }

```

Output:

A

65

66

.....

Q9.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 5;
5     int y = x++ + ++x;
6     cout << y << endl;
7     cout << x << endl;
8     return 0;
9 }

```

Output:

12

7

Explanation:

Note: Modifying a variable multiple times in a single statement (like `x++ + ++x`) is technically considered “Undefined Behavior” in C++, meaning different compilers might theoretically give different results! However, based on the output 12, here is how standard compilers evaluate it step-by-step:

- `x++` (post-increment) uses the current value (5) for the calculation, then increases `x` to 6 behind the scenes.
- `++x` (pre-increment) takes the new `x` (6), immediately increases it to 7, and uses 7 for the calculation.
- So, $y = 5 + 7 = 12$. The final modified value of `x` is 7.

.....

Q10.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     double x = 5.9;
5     int y = (int)x;
6     cout << y << endl;
7     cout << (int)3.99 << endl;
8     return 0;
9 }

```

Output:

5

3

.....

For Q11 to Q20: Write clean, compilable C++ programs. Use appropriate data types, *cin* for input, and *cout* for output.

Q11. Take an integer n as input. Print the value of $4n + 9$.

Sample — Input: 5 $\Rightarrow$ Output: 29

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6     cout << (4 * n + 9) << endl;
7     return 0;
8 }
```

.....

Q12. Take four integers a, b, c, d as input. Compute and print $(a + b) \times (c + d)$.

Sample — Input: 1 2 3 4 $\Rightarrow$ Output: 21

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int a, b, c, d;
5     cin >> a >> b >> c >> d;
6     cout << (a + b) * (c + d) << endl;
7     return 0;
8 }
```

.....

Q13. Write a program to calculate the **Volume of a Sphere** given radius r . Use $\pi = 3.14$.

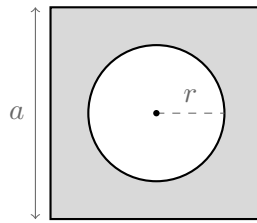
$$V = \frac{4}{3} \pi r^3$$

Sample — Input: 3 $\Rightarrow$ Output: 113.04

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     double r;
5     cin >> r;
6     double V = (4.0 / 3.0) * 3.14 * r * r * r;
7     cout << V << endl;
8     return 0;
9 }
```

.....

Q14. A circle of radius r is inside a square of side a (the circle always sits completely inside the square without touching its edges). Find the area of the **shaded region** — the part inside the square but outside the circle. Use $\pi = 3.14$.



Sample — Input: $r = 2$, $a = 6 \Rightarrow$ Output: 23.44

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     double r, a;
5     cin >> r >> a;
6     double areaSquare = a * a;
7     double areaCircle = 3.14 * r * r;
8     cout << areaSquare - areaCircle << endl;
9     return 0;
10 }
```

.....

Q15. Take a char as input and print its ASCII value.

Sample — Input: Z $\Rightarrow$ Output: 90

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     char ch;
5     cin >> ch;
6     cout << (int)ch << endl;
7     return 0;
8 }
```

.....

Q16. Take an integer n as input. Print the minimum non-negative integer that must be added to n to make it exactly divisible by 7. Use the modulus operator.

Sample — Input: 20 $\Rightarrow$ Output: 1 (since $20 + 1 = 21 = 7 \times 3$)

Explanation:

- We first find the **remainder** of n divided by 7. For $n = 20$, the remainder is 6. To make it exactly divisible, we need to add the missing part: $7 - \text{remainder}$ (which is $7 - 6 = 1$).
- **What if n is already divisible by 7?** (e.g., $n = 14$). The remainder is 0. Doing $7 - 0$ gives 7. But if we add 7, we skip ahead to the *next* multiple (21). The problem asks for the *minimum* non-negative integer, which should be 0.
- This is why we apply an extra $\% 7$ at the very end: $(7 - \text{remainder}) \% 7$. If the remainder is 0, $(7 - 0) \% 7$ safely becomes 0, giving us the correct answer!

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6     int remainder = n % 7;
```

```

7     int toAdd = (7 - remainder) % 7;
8     cout << toAdd << endl;
9     return 0;
10  }

```

Q17. Take two integers a and b ($b \neq 0$) as input. Print the result of **ceiling integer division** of a by b .

Ceiling division always rounds up — e.g., $\lceil 7/2 \rceil = 4$, whereas normal integer division gives 3. Hint - Formula for positive integers: $(a + b - 1) / b$.

Sample — Input: 7 2 $\Rightarrow$ Output: 4

Explanation:

- Normal integer division (a / b) always rounds *down* (truncates the decimal part).
- To simulate rounding *up* (ceiling division) without converting to floats, we use the math trick $(a + b - 1) / b$.
- **Case 1 (Exact multiple):** Let $a = 6, b = 3$. Formula: $(6 + 3 - 1) / 3 = 8 / 3 = 2$. It correctly stays at 2 without overshooting.
- **Case 2 (Has remainder):** Let $a = 7, b = 3$. Formula: $(7 + 3 - 1) / 3 = 9 / 3 = 3$. It correctly rounds up! The $(b - 1)$ adds just enough so that any remainder pushes the numerator into the next multiple of b .
- **Crucial Note:** This specific formula assumes we are dealing **only with positive numbers**. If a or b can be negative, this trick fails due to how C++ handles division and remainders for negative integers.

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a, b;
5      cin >> a >> b;
6      cout << (a + b - 1) / b << endl;
7      return 0;
8  }

```

Q18. Compute the **Kinetic Energy** of a moving object. Take the Mass m (in kg, double) and Velocity v (in m/s, double) as input from the user. Be careful about integer vs. double division when applying the fraction coefficient.

$$KE = \frac{1}{2} \times m \times v^2$$

Sample — Input: $m = 10, v = 4 \Rightarrow$ Output: 80

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      double m, v;
5      cin >> m >> v;
6      double ke = 0.5 * m * v * v;
7      cout << ke << endl;
8      return 0;
9  }

```

.....

Q19. Take an amount in **Rupees** (double) and convert it to **Dollars**, given that **1 Dollar = 100 Rupees**.

Sample — Input: 250 $\Rightarrow$ Output: 2.5

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     double rupees;
5     cin >> rupees;
6     cout << rupees / 100.0 << endl;
7     return 0;
8 }
```

.....

Q20. A shopkeeper applies a discount of $d\%$ on the marked price m , and then charges a tax of $t\%$ on the discounted price. Take m , d , and t (all doubles) as input and print the final amount the customer pays.

$$\text{Discounted Price} = m - \frac{d}{100} \times m \quad \text{Final Price} = \text{Discounted Price} \times \left(1 + \frac{t}{100}\right)$$

Sample — Input: $m = 1000$, $d = 10$, $t = 18 \Rightarrow$ Output: 1062

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     double m, d, t;
5     cin >> m >> d >> t;
6     double discountedPrice = m - (d / 100.0) * m;
7     double finalPrice = discountedPrice * (1.0 + (t / 100.0));
8     cout << finalPrice << endl;
9     return 0;
10 }
```

.....

All the best! Keep Coding ♡